

UNIVERSITÀ DEGLI STUDI DI PAVIA
CORSO DI INGEGNERIA DEL SOFTWARE

Documento di Progettazione

PM_Collegio v2 : Sistema di gestione prenotazioni camere
per collegi universitari

Diagrammi UML e diagramma ER del livello di persistenza

Progetto	PM_Collegio v2
Autore	Giresse Kengne
Versione documento	1.0
Data	11 giugno 2026
Documenti correlati	SPECIFICA_REQUISITI.pdf (SRS v1.0)

1. Introduzione

Questo documento raccoglie la **documentazione di progettazione** del sistema PM_Collegio v2 : l'architettura, i diagrammi UML (casi d'uso, classi, sequenza, stati) e il diagramma ER del livello di persistenza. Completa la Specifica dei Requisiti (SRS) : dove la SRS dice *cosa* il sistema deve fare, questo documento descrive *come* è strutturato per farlo.

I diagrammi sono stati ricavati dal codice sorgente del progetto (package `it.collegio.*`) e dallo schema MySQL di riferimento ; ogni identificativo di requisito citato (RF-n, RNF-n, RI-n) rimanda alla SRS versione 1.0.

1.1 Contenuto

- **Cap. 2** : architettura a strati MVC + DAO + DTO (diagramma dei package).
- **Cap. 3** : diagramma dei casi d'uso, con attori AS, AC, AR, U.
- **Cap. 4** : diagrammi delle classi (modello di dominio compatto + spaccato MVC del sottosistema Fatture).
- **Cap. 5** : diagrammi di sequenza dei quattro flussi chiave (login, check-in, check-out, pagamento).
- **Cap. 6** : diagrammi di stato di Reservation, Fattura e Room.
- **Cap. 7** : progettazione della persistenza, diagramma ER e tabella delle relazioni.
- **Cap. 8** : tracciabilità dei diagrammi verso i requisiti.

2. Architettura del sistema

PM_Collegio v2 adotta un'architettura a strati ispirata al pattern **MVC**, integrata con **DAO** per l'accesso ai dati e **DTO** per il trasporto di dati joinati. La regola fondamentale (RNF-1) è la direzione delle dipendenze : una View non parla mai con il database, ma solo con un Controller ; il Controller orchestra i DAO ; solo i DAO eseguono SQL, prelevando le query da `QueryContainer` (RNF-2).

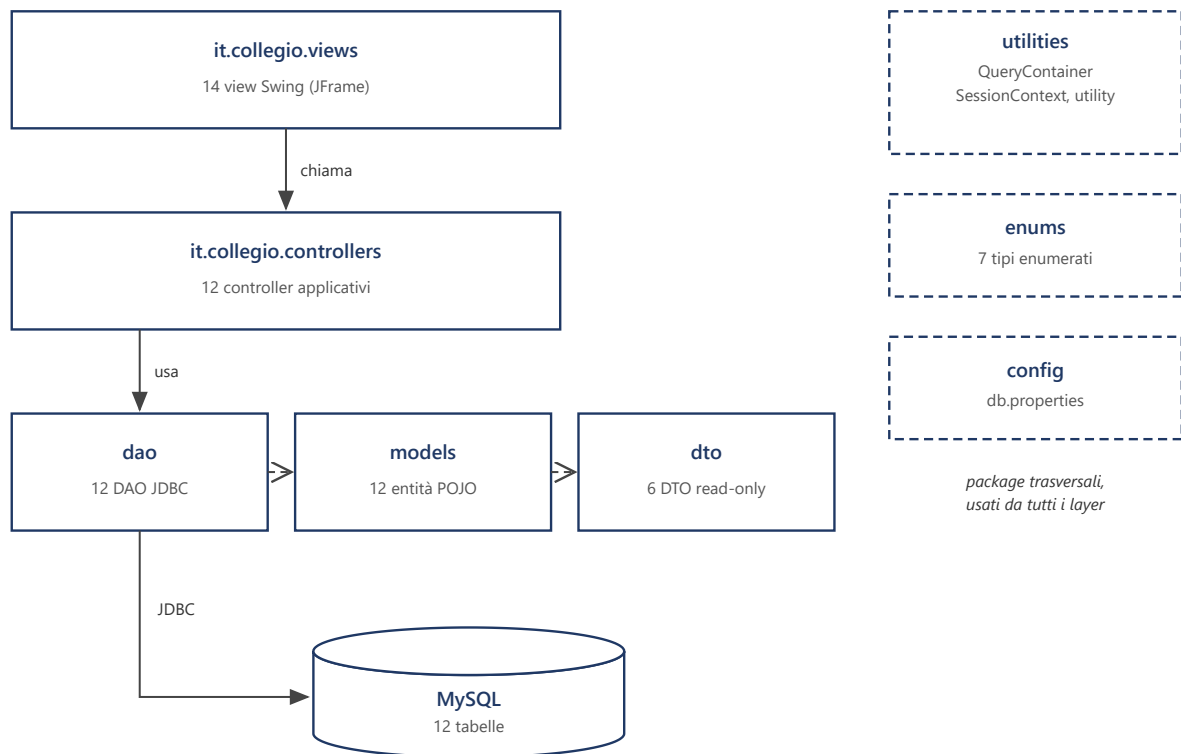
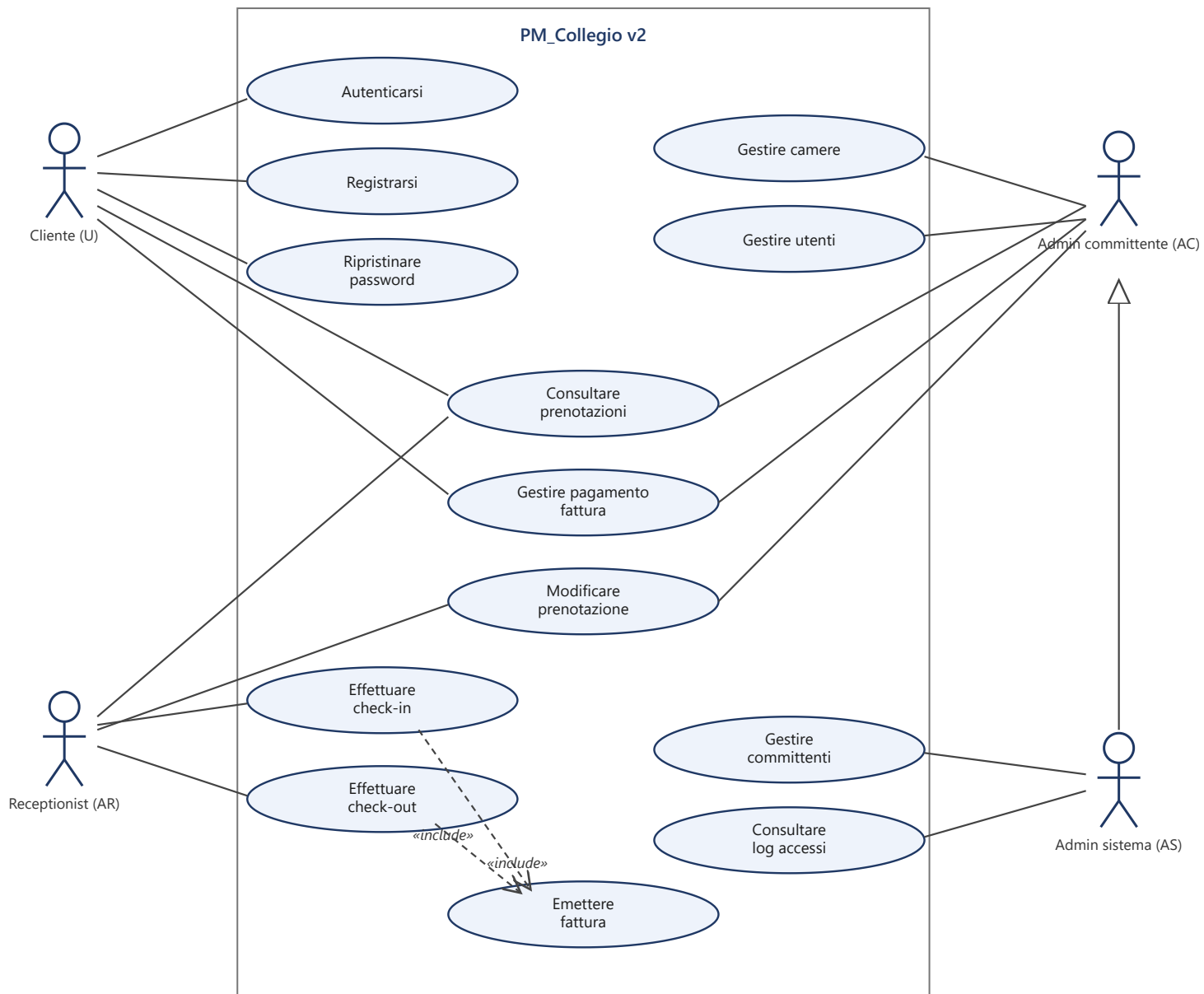


Fig. 2.1 : diagramma dei package, con la direzione delle dipendenze tra layer.

I package `utilities`, `enums` e `config` sono trasversali : `QueryContainer` centralizza le query SQL, `SessionContext` mantiene lo stato della sessione corrente, i tipi enumerati codificano gli stati del dominio (camera, prenotazione, fattura, utente), `db.properties` esternalizza le credenziali (RNF-8, letto da `DatabaseConnection` all'avvio).

3. Diagramma dei casi d'uso

Gli attori sono i quattro ruoli del sistema : Cliente (U), Receptionist (AR), Admin committente (AC), Admin di sistema (AS). La generalizzazione $AS \rightarrow AC$ indica che l'admin di sistema può svolgere tutte le operazioni dell'admin committente, su qualunque committente.



*Tutti i casi d'uso operativi richiedono l'autenticazione («include» Autenticarsi, omissso per leggibilità).
Il ruolo U accede in sola lettura alle proprie prenotazioni (RF-17) e gestisce solo le proprie fatture (RF-27).*

Fig. 3.1 : diagramma dei casi d'uso. Il check-in e il check-out includono l'emissione della fattura (RF-21, RF-26).

3.1 Matrice attori / casi d'uso

Caso d'uso	U	AR	AC	AS	Requisiti
Autenticarsi (login / logout)	✓	✓	✓	✓	RF-1 ... RF-6
Registrarsi	✓				RF-7 ... RF-10
Ripristinare password	✓	✓	✓	✓	RF-11
Consultare prenotazioni	✓ (proprie, sola lettura)	✓	✓	(✓ via AC)	RF-12 ... RF-14, RF-17
Modificare prenotazione (con storico)		✓	✓	(✓ via AC)	RF-15, RF-16
Effettuare check-in		✓			RF-19 ... RF-22
Effettuare check-out		✓			RF-23 ... RF-26
Gestire pagamento fattura (paga / annulla)	✓ (proprie)		✓	(✓ via AC)	RF-27 ... RF-30
Gestire camere			✓	(✓ via AC)	RF-31
Gestire utenti			✓	(✓ via AC)	RF-32
Gestire committenti				✓	RF-33
Consultare log accessi				✓	RF-35

La visibilità delle funzioni in interfaccia è governata da `configureButtonsForRole()` nella HomeView (RF-36) ; i filtri dati per il ruolo U sono applicati server-side nelle query (RNF-7).

4. Diagrammi delle classi

4.1 Modello di dominio (vista compatta)

Le 12 classi del package `it.collegio.models` sono POJO in corrispondenza 1:1 con le tabelle del database (RNF-3). La vista è compatta : il dettaglio degli attributi è nel diagramma ER del Cap. 7.

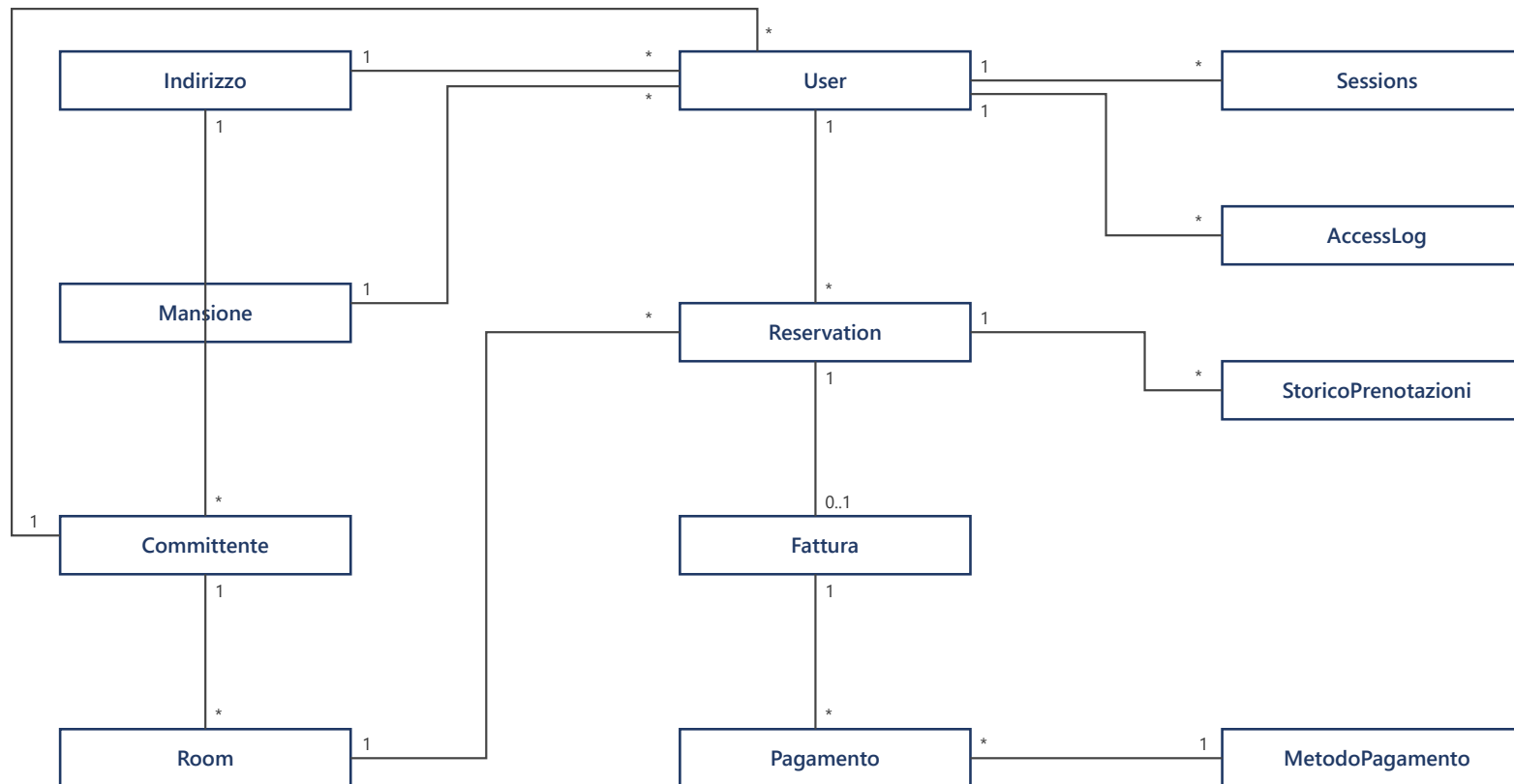


Fig. 4.1 : modello di dominio. Per leggibilità sono omesse le associazioni multi-committenza di Room, Reservation e Sessions verso Committente, Sessions verso Mansione, StoricoPrenotazioni verso User e il ruolo «gestore» di Committente verso User : sono tutte riportate nel diagramma ER (Fig. 7.1).

4.2 Spaccato MVC : il sottosistema Fatture

Lo spaccato mostra come un sottosistema attraversa i layer : due view condividono lo stesso controller, che orchestra tre DAO ; i dati joinati viaggiano nel DTO `FatturaDettaglio` , che include `reservationStato` per la regola di pagabilità combinata (RF-30).

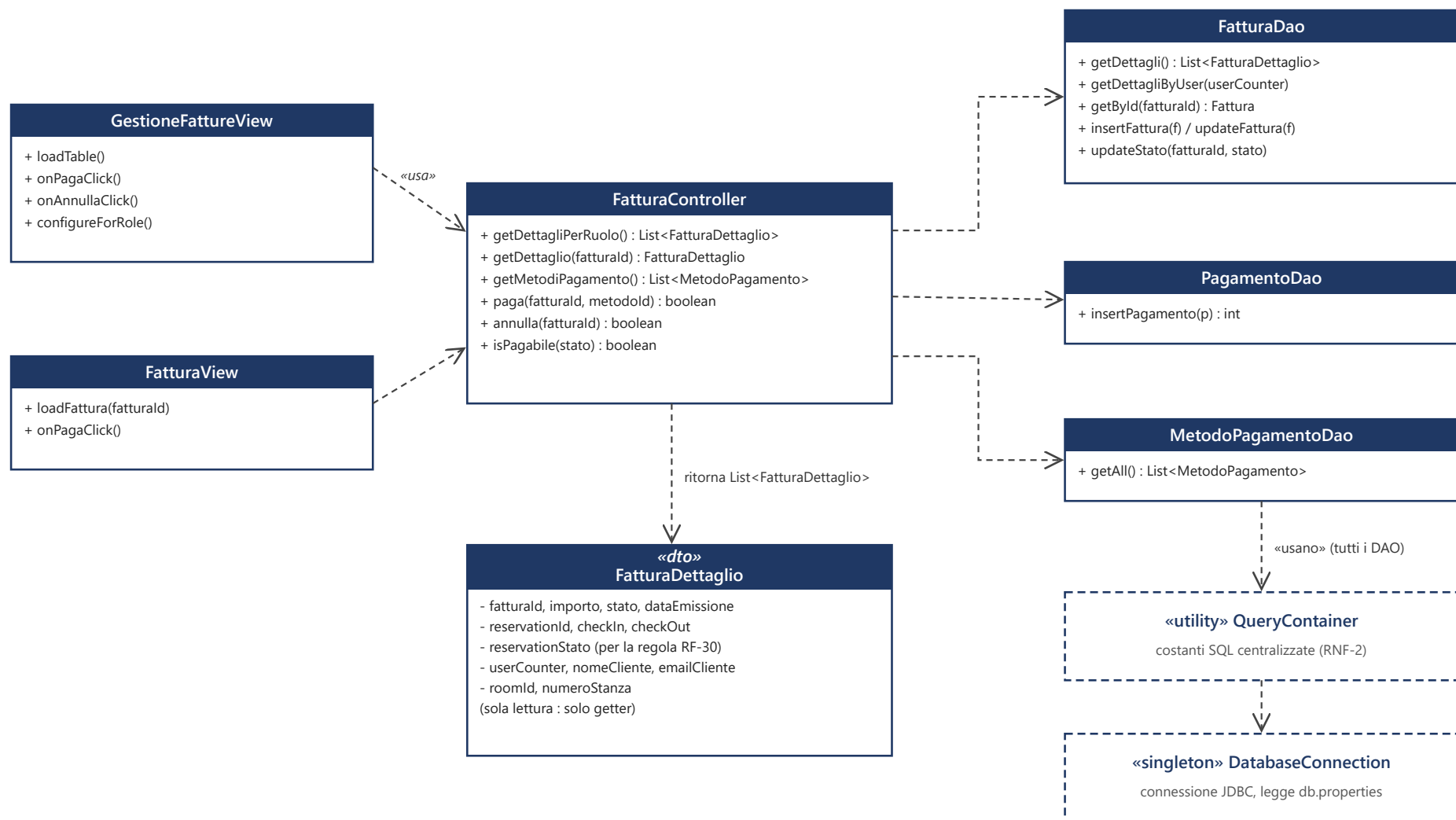


Fig. 4.2 : spaccato del sottosistema Fatture. I DAO costruiscono i DTO dai ResultSet delle query joinate ; lo stesso schema (View → Controller → DAO → DTO) si ripete in tutti i sottosistemi.

5. Diagrammi di sequenza

5.1 Login (RF-1 ... RF-5)

Il login è il flusso più articolato : valida le credenziali, verifica lo stato dell'account, registra l'accesso, crea la sessione con scadenza a 8 ore e popola il contesto applicativo.

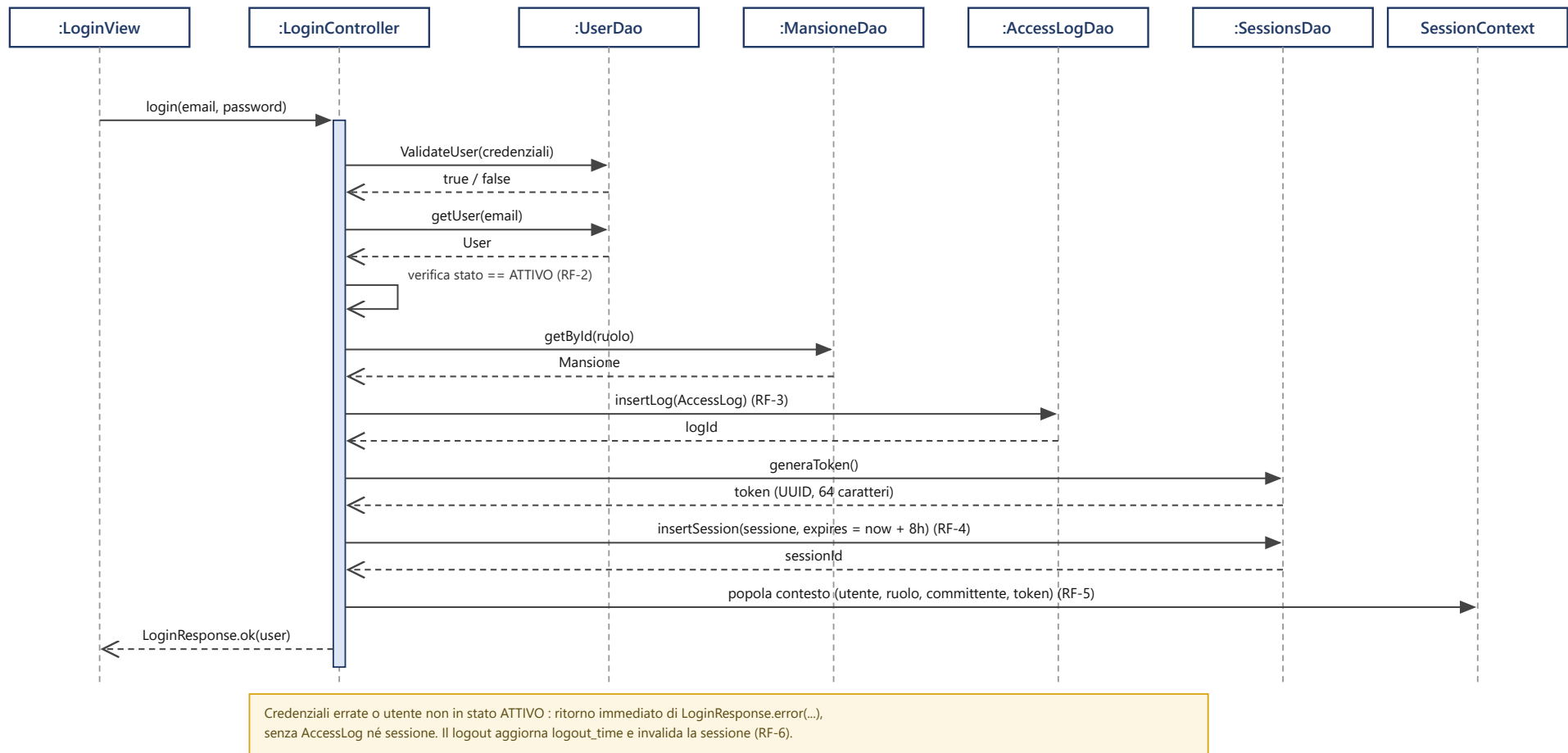


Fig. 5.1 : sequenza del login (LoginController.login).

5.2 Check-in (RF-19 ... RF-22)

Il check-in è una transazione a tre passi : prenotazione, fattura iniziale a importo zero, occupazione della camera. Il frame indica il confine transazionale (RNF-5).

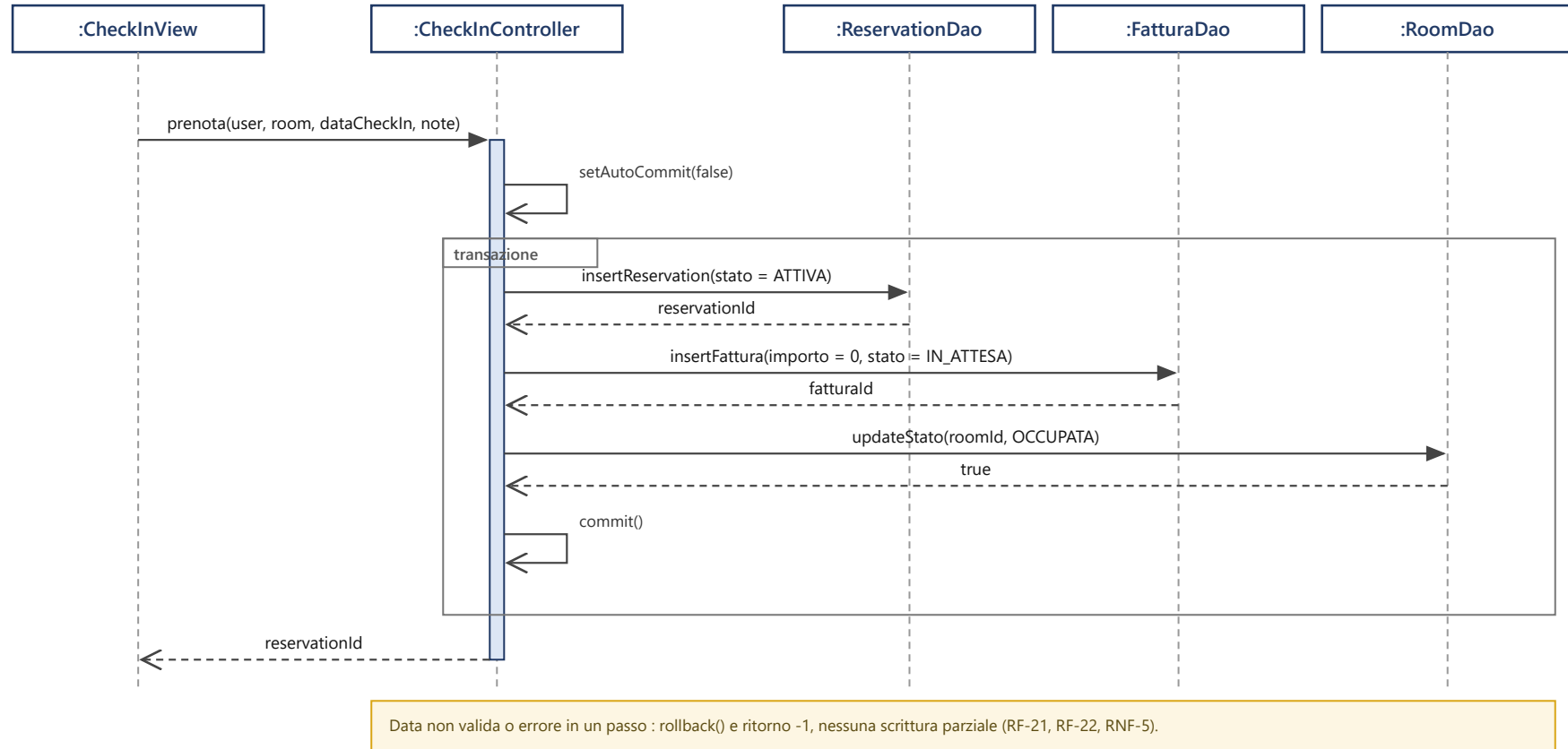


Fig. 5.2 : sequenza del check-in transazionale (CheckInController.prenota).

5.3 Check-out (RF-23 ... RF-26)

Il check-out avviene in due tempi : prima l'anteprima con il calcolo server-side dell'importo, poi la conferma che esegue la transazione di chiusura.

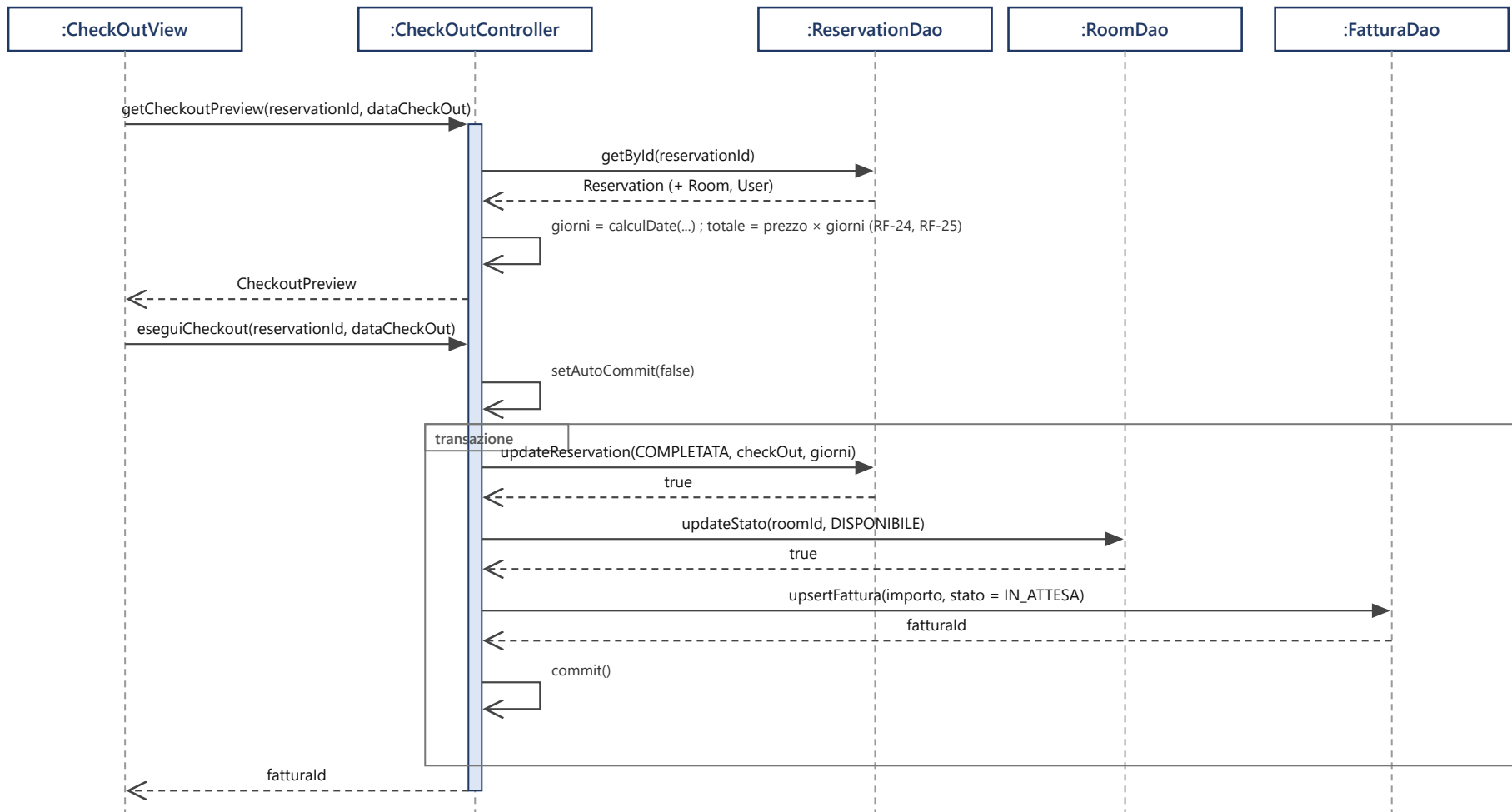


Fig. 5.3 : sequenza del check-out (CheckOutController). L'upsert aggiorna la fattura creata al check-in, o ne crea una nuova se assente.

5.4 Pagamento fattura (RF-28, RF-30)

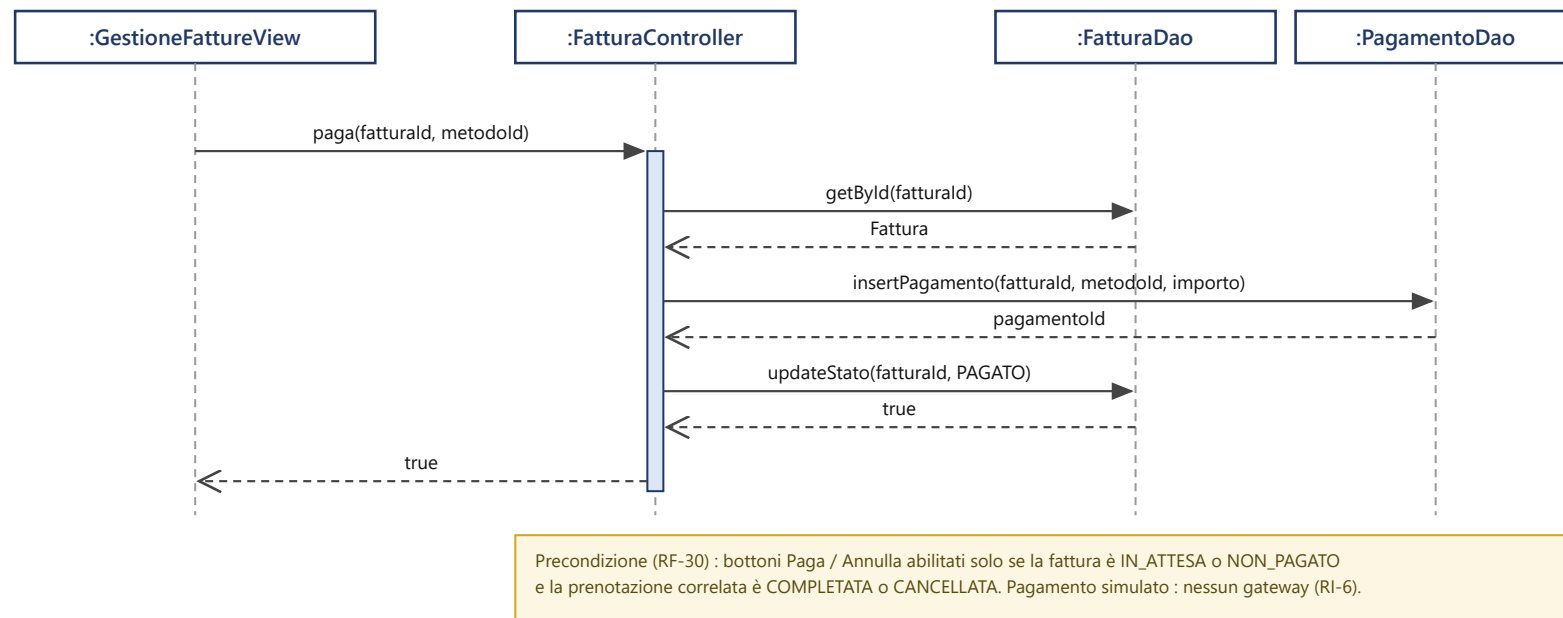


Fig. 5.4 : sequenza del pagamento simulato (FatturaController.paga).

6. Diagrammi di stato

Le tre macchine a stati del dominio corrispondono ai campi ENUM dello schema : sono gli stati che i flussi dei capitoli precedenti leggono e modificano. Nessuna transizione avviene fuori da quelle mostrate.

6.1 Reservation (status)

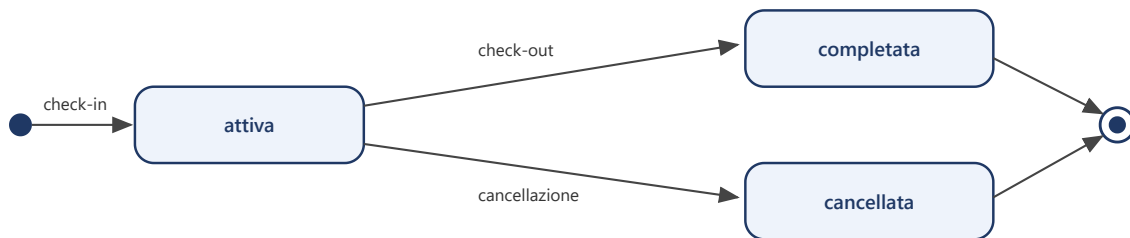


Fig. 6.1 : ciclo di vita di una prenotazione. La modifica date (RF-15, RF-16) non cambia lo stato : agisce solo su una prenotazione attiva e traccia in StoricoPrenotazioni.

6.2 Fattura (stato)

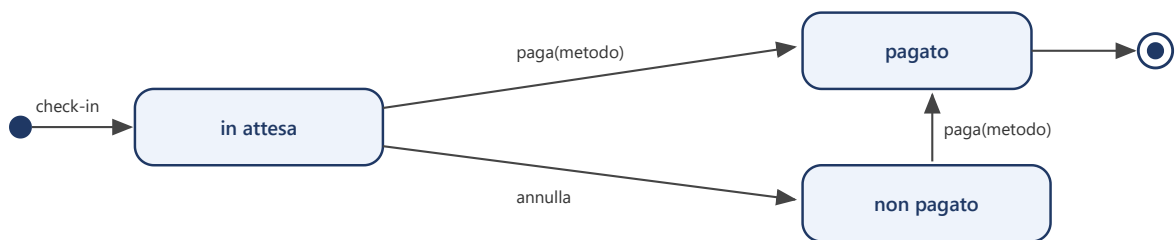


Fig. 6.2 : ciclo di vita di una fattura. Le transizioni paga / annulla sono soggette alla regola combinata RF-30 ; il check-out riporta la fattura a «in attesa» con l'importo ricalcolato.

6.3 Room (stato)

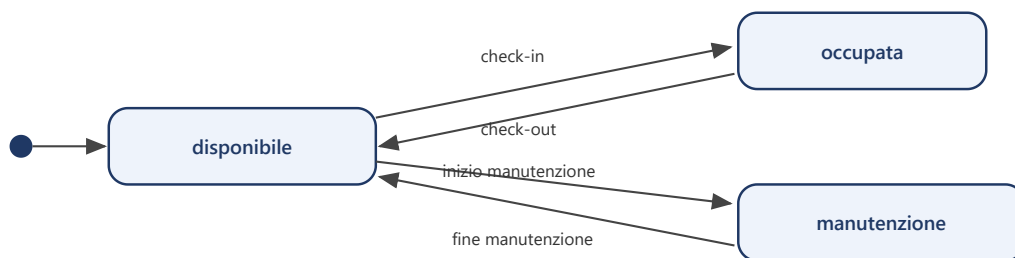


Fig. 6.3 : stati di una camera. Solo le camere disponibili compaiono nella lista del check-in (RF-19) ; le transizioni di manutenzione avvengono da ManageRoom (RF-31).

7. Progettazione della persistenza : diagramma ER

Il livello di persistenza è un database MySQL con 12 tabelle. Il diagramma riporta tutte le entità con chiavi primarie (sottolineate), chiavi esterne (corsivo, FK) e le relazioni con cardinalità. Le colonne ENUM corrispondono alle macchine a stati del Cap. 6.

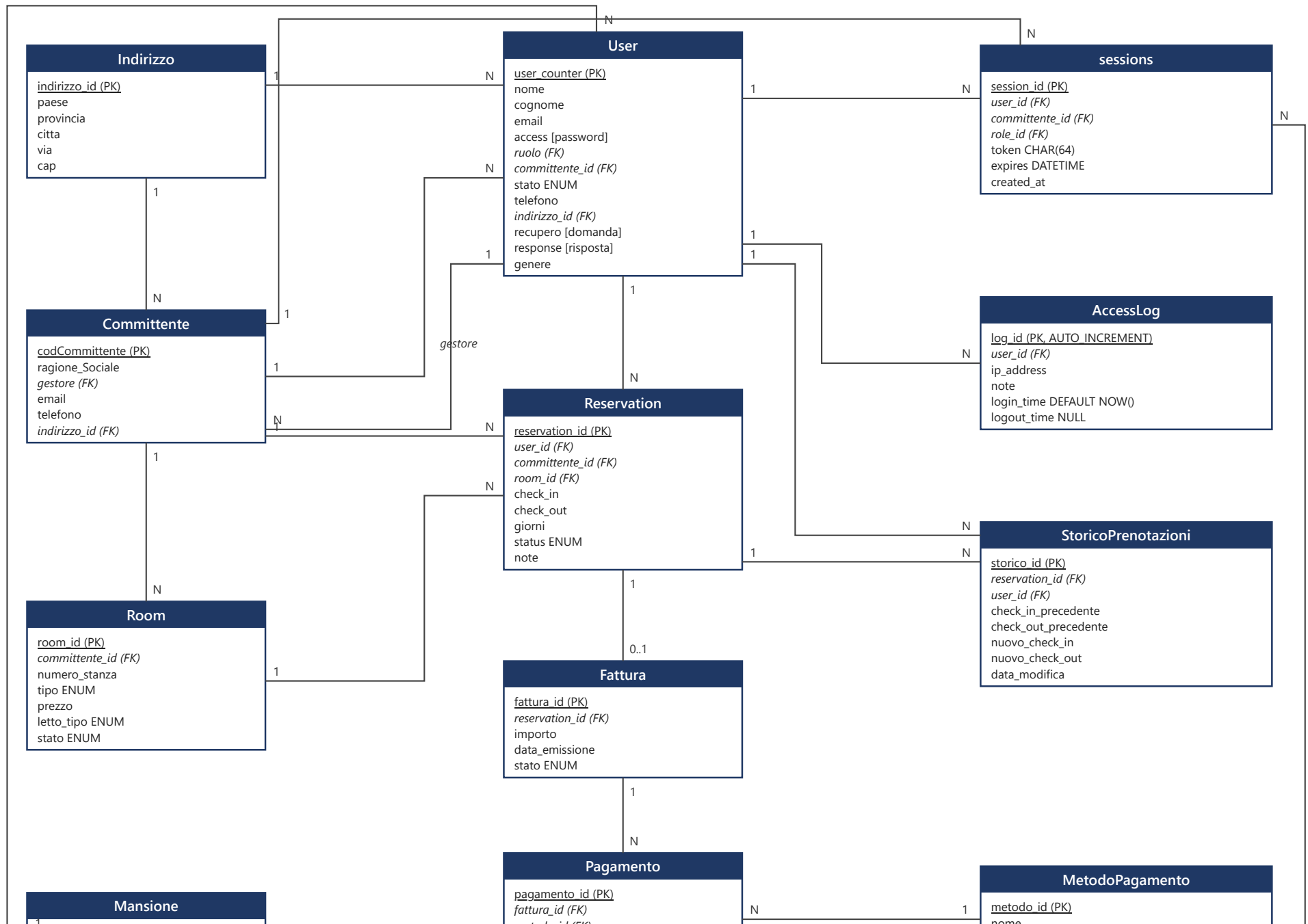




Fig. 7.1 : diagramma ER del livello di persistenza (notazione relazionale, PK sottolineate, FK in corsivo).

7.1 Tabella delle relazioni

Relazione (N : 1)	Chiave esterna	Significato
User → Mansione	User.ruolo	Ogni utente ha esattamente un ruolo (AS, AC, AR, U)
User → Committente	User.committente_id	Ogni utente appartiene a un collegio
User → Indirizzo	User.indirizzo_id	Indirizzo anagrafico dell'utente
Committente → Indirizzo	Committente.indirizzo_id	Sede del collegio
Committente → User	Committente.gestore	Utente gestore del collegio (relazione 1 : N invertita)
Room → Committente	Room.committente_id	Ogni camera appartiene a un collegio (multi-committenza, RNF-17)
Reservation → User / Room / Committente	user_id , room_id , committente_id	Una prenotazione lega un cliente a una camera di un collegio
Fattura → Reservation	Fattura.reservation_id	Una fattura per prenotazione (0..1, creata al check-in, aggiornata al check-out)
Pagamento → Fattura / MetodoPagamento	fattura_id , metodo_id	Pagamento simulato con metodo scelto (RF-28)
AccessLog → User	AccessLog.user_id	Audit degli accessi (RF-3, RNF-11)
sessions → User / Committente / Mansione	user_id , committente_id , role_id	Sessione applicativa con token e scadenza (RF-4, RNF-9)
StoricoPrenotazioni → Reservation / User	reservation_id , user_id	Audit delle modifiche alle prenotazioni (RF-16)

7.2 Note di progettazione della persistenza

- **Chiave applicativa** : `User.user_counter` è VARCHAR(100) nel formato `xxx{id}{NOME}` , generata da `UserDao.generateUniqueCounter()` con retry su collisione (RF-10). Non è un AUTO_INCREMENT.
- **ENUM ↔ enums Java** : le colonne ENUM di MySQL (stato camera, prenotazione, fattura, utente) sono mappate 1:1 sui tipi del package `it.collegio.enums` ; la conversione avviene nei DAO.
- **Vincolo logico fattura** : la relazione Fattura-Reservation è 0..1 applicativa, garantita dall'upsert del check-out, non da un vincolo UNIQUE sul DB.
- **Multi-committenza** : `committente_id` è presente su User, Room, Reservation e sessions (RNF-17) ; l'esercizio attuale usa il solo committente 1 (VG-4).
- **Credenziali** : i parametri di connessione vivono in `db.properties` fuori dal repository (RNF-8) ; `DatabaseConnection` li carica dal classpath in un blocco statico.
- **Dati di seed** : lo script DDL include mansioni, indirizzi, committente 1, utenti e metodi di pagamento, presupposti dal sistema al primo avvio (P-2 ... P-5 della SRS).

8. Tracciabilità dei diagrammi

Diagramma	Copre i requisiti	Componenti rappresentati
Fig. 2.1 Package	RNF-1 ... RNF-4, RNF-8	Struttura dei package <code>it.collegio.*</code>
Fig. 3.1 Casi d'uso	RF-1 ... RF-36 (vista d'insieme), RF-36	Attori e funzionalità
Fig. 4.1 Classi (dominio)	RNF-3	Package <code>models</code>
Fig. 4.2 Classi (slice Fatture)	RF-27 ... RF-30, RNF-1, RNF-2	FatturaController, FatturaDao, PagamentoDao, MetodoPagamentoDao, FatturaDettaglio
Fig. 5.1 Sequenza login	RF-1 ... RF-6, RNF-5	LoginController e DAO collegati
Fig. 5.2 Sequenza check-in	RF-19 ... RF-22, RNF-5	CheckInController
Fig. 5.3 Sequenza check-out	RF-23 ... RF-26, RNF-5, RNF-6	CheckOutController
Fig. 5.4 Sequenza pagamento	RF-28 ... RF-30	FatturaController
Fig. 6.1 ... 6.3 Stati	RF-21, RF-26, RF-28 ... RF-31	Enums ReservationStatus, FatturaStatus, RoomStatus
Fig. 7.1 ER	RNF-17, RF-3, RF-4, RF-16	Le 12 tabelle MySQL

Il documento di progettazione è coerente con la SRS v1.0 e con il codice del repository `PM_Collegiov2` alla data dell'11 giugno 2026.